

دفترچه خاطرات

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: ساده
- طراح: دانیال یاقوتی

چند روزی از پایان جنگ می‌گذرد که مراد تصمیم می‌گیرد از روستای مرادآباد به شهرش باز گردد. از بخت بد مراد، زمانی که به خانه باز می‌گردد متوجه می‌شود لوله های همسایه طبقه بالا دچار نشتی شده و حالا تمام انباریش را آب گرفته. برچسب مواد غذایی که به خاطر تماس با آب و رطوبت حالا دیگر مخدوش و غیر قابل خواندن شده، مراد را نگران کرده که مبادا غذا هایش فاسد شده باشد. اما مراد که همیشه عاشق ثبت خاطراتش بوده راه‌حلی به ذهنش می‌رسد! مراد در دفترچه خاطراتش هر وقت که خوراکی را خریداری می‌کند با فرمت `yyyy-mm-dd:Item` ثبت می‌کند. حالا مراد از شما کمک می‌خواهد تا برنامه‌ای بنویسید که با بررسی دفترچه خاطرات او تعداد مواد غذایی که هنوز فاسد نشده‌اند را برایش پیدا کنید.

ورودی

خطوط ورودی به ترتیب و شرح زیر است:

-خط اول: ورودی یک رشته شامل لیست مواد غذایی که مراد دنبال بررسی تاریخ انقضایشان می‌گردد، به فرمت `Item_1 Item_2 ... Item_N` داده می‌شود:

Example:

Milk Egg Banana

که در آن هر آیتم با یک فاصله(Space)از دیگری جدا شده. همچنین تضمین می‌شود:

$$N \leq 20$$

-خط دوم: یک رشته شامل تاریخ روز حال در دنیای مراد به فرمت `yyyy-mm-dd` داده می‌شود.

-خط سوم: یک رشته شامل بخشی از دفترچه خاطرات مراد به شما داده می‌شود.

شما وظیفه دارید رجکسی طراحی کنید تا بخش هایی از متن خاطرات با فرمت yyyy-mm-dd:Item را استخراج کند که در آن اولاً تاریخ ثبت شده **یک تاریخ معتبر** باشد، ثانياً آیتم مقابل تاریخ یکی از **آیتم های موجود در لیست خط اول ورودی** باشد. توجه کنید که **رجکس شما نباید حساس به حروف کوچک یا بزرگ باشد**.

▼ راهنمایی

تاریخ معتبر تاریخی است که عدد ماه آن یکی از اعداد بازه [1,12] باشد؛ همچنین با توجه به نیمه اول یا دوم سال، ماکزیمم روز ثبت شده در آن 31 یا 30 روز باشد.

Example:

1398-7-29 // Valid

1404-11-31 // Not Valid

هر سال را 366 روز در نظر بگیرید. **نیازی به بررسی کبیسه بودن یا نبودن سال نیست.**

در نهایت با استخراج قسمت های مورد نیاز باید بررسی کنید آیا خوراکی استخراج شده فاسد شده یا خیر. توجه داشته باشید که ممکن است از یک خوراکی چند مورد سالم پیدا و یا هیچ موردی یافت نشود.

خروجی

خروجی شما باید به ترتیب آیتم های موجود در لیست و با فرمت زیر باشد؛ به طوری که در مقابل هر آیتم تعداد موارد غیر فاسد پیدا شده از آن آیتم قرار می‌گیرد:

```
=====Items=====
Item_1: 5
Item_2: 0
.
.
Item_N: 2
=====
```

مثال 1

ورودی

Milk Egg Banana

1399-6-25

Today is sunny.I got EXCELLENT 1399-5-25:egg and also 1399-12-7:Milk.. Wow! there i

خروجی

```
=====Items=====
Milk: 1
Egg: 0
Banana: 0
=====
```

توضیح خروجی

تاریخ مصرف اولین تخم مرغ(egg) گذشته. تاریخ مصرف اولین شیر(Milk) هنوز سر نرسیده. تاریخ مصرف اولین موز(Banana) یک تاریخ نامعتبر است زیرا عدد ماه برابر 13 می باشد.

مثال 2

ورودی

Milk PizZA bread

255-7-7

I love 255-7-7:Bread! hmm.. 255-2-31:PIZAA it doesn't seem fresh but 255-7-8:piZZa

خروجی

```
=====Items=====
Milk: 0
PizZA : 1
bread: 1
=====
```

توضیح خروجی

تاریخ مصرف اولین نان (Bread) تا آخر روز حال معتبر است. تاریخ مصرف اولین پیتزا گذشته و تاریخ دومین پیتزا هنوز معتبر است. هیچ شیری (Milk) مطابق با فرمت لازم در متن یافت نمی‌شود.

-همچنین دقت کنید اسامی مواد غذایی ورودی و خروجی **کاملاً یکسان** است.

فرزاد کوهنورد

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: متوسط
- طراح: محمد امین مولائی

فرزاد کوهنوردی را به تازگی شروع کرده است و هنوز مبتدی است. برای همین میخواهد قبل از کوهنوردی ارتفاع نقاط مختلف کوهستان را بررسی کند و در نهایت امن ترین قله را برای صعود انتخاب کند. فرزاد آرایه ای شامل n نقطه از ارتفاع نقاط کوهستان را در اختیار دارد. ارتفاع نقطه i را با $h[i]$ نمایش میدهیم.

نقطه i از کوهستان امن محسوب میشود اگر:

۱. ارتفاع آن نقطه ($h[i]$) از بیشینه ارتفاع نقاط سمت چپش **بزرگتر** باشد.
۲. ارتفاع آن نقطه از کمینه (مینیمم) ارتفاع نقاط سمت راستش **کوچکتر** باشد.

همچنین دقت کنید که نقاط ابتدایی و انتهایی آرایه جزو قله های امن حساب **نمیشوند**. فرزاد از بین قله های امن کوهستان دنبال **کوتاهترین** قله است. به او کمک کنید تا بتواند این آخر هفته کوهنوردی را شروع کند!

ورودی

در خط اول n داده میشود.

در خط دوم n عدد از $h[0]$ تا $h[n - 1]$ با فاصله داده میشوند.

$$1 \leq n \leq 10^5$$

$$\forall i \quad 1 \leq h[i] \leq 10^5$$

خروجی

در خط اول تعداد قله های امن را چاپ کنید.

در خط دوم ارتفاع کوتاهترین قله امن را چاپ کنید.

اگر هیچ قله امنی وجود نداشت 1- را چاپ کنید.

مثال

ورودی نمونه 1

```
10
3 5 2 6 4 7 8 9 10 11
```

خروجی نمونه 1

```
4
7
```

ورودی نمونه 2

```
5
5 4 3 2 1
```

خروجی نمونه 2

```
-1
```

باقر و چراغ‌ها

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: متوسط
- طراح: حدیث صابری

باقر قصد دارد مسیری به طول L کیلومتر را با دوچرخه طی کند. در طول این مسیر، N چراغ راهنمایی وجود دارد که هر کدام از آنها چرخه‌ای دارند. چراغ قرمز i ام در فاصله‌ی d_i کیلومتری خانه‌ی باقر قرار دارد و در هر چرخه r_i دقیقه قرمز است و g_i دقیقه سبز است و s_i وضعیت اولیه چراغ i در زمان $t = 0$ (0 برای قرمز، 1 برای سبز) را نشان می‌دهد.

قوانین حرکت و چراغ‌ها:

۱. **سرعت باقر:** باقر با سرعت ثابت 1 کیلومتر بر دقیقه حرکت می‌کند.
۲. **زمان شروع:** باقر در زمان $t = 0$ از نقطه شروع (کیلومتر 0) حرکت می‌کند.

۳. قانون چراغ‌ها:

- وقتی باقر به چراغ i می‌رسد، وضعیت آن چراغ در **لحظه رسیدن** او بررسی می‌شود.
- اگر چراغ در لحظه رسیدن سبز باشد، باقر بدون توقف عبور می‌کند.
- اگر چراغ در لحظه رسیدن قرمز باشد، باقر **دقیقا به اندازه مدت زمانی که طول می‌کشد تا چراغ سبز شود، توقف می‌کند.** پس از سبز شدن، بلافاصله حرکت خود را ادامه می‌دهد.
- چرخه چراغ‌ها تکراری است؛ یعنی پس از r_i دقیقه قرمز بودن، g_i دقیقه سبز می‌شود و سپس دوباره r_i دقیقه قرمز و ...
- وضعیت اولیه‌ی s_i مشخص می‌کند که در $t = 0$ ، چراغ در کدام بخش از چرخه خود قرار دارد. اگر s_i ، 0 باشد، یعنی هنگام شروع سفر باقر، این چراغ تازه قرمز شده است و اگر 1 باشد یعنی هنگام شروع سفر باقر، این چراغ تازه سبز شده است.

شما باید برنامه‌ای بنویسید که با دریافت اطلاعات چراغ‌ها، دو خروجی اصلی زیر را محاسبه و چاپ کند:

۱. **زمان کل سفر (T):** مجموع زمان حرکت و زمان توقف‌های باقر تا رسیدن به انتهای مسیر (زمان شروع تا پایان مسیر باقر).

۲. **بیشترین زمان توقف (MaxWait):** طولانی‌ترین مدتی که باقر جلوی یک چراغ مجبور به توقف بوده است. اگر هیچ توقفی رخ نداده باشد، مقدار آن 0 است.

دقت داشته باشید که اطلاعات چراغ‌ها ممکن است بر اساس فاصله ی d_i ، به شما داده نشده باشد اما باقر همیشه از نزدیک‌ترین چراغ به دورترین چراغ حرکت می‌کند.

ورودی

در خط اول N و L آمده است.

در هر یک از N خط بعد، در خط اُم، به ترتیب d_i ، r_i ، g_i و s_i با یک فاصله آمده است.

$$1 \leq N, r_i, g_i \leq 100$$

$$1 \leq d_i < l \leq 1000$$

تضمین می‌شود که تمامی d_i ها متمایز و تمامی اعداد ورودی صحیح‌اند.

خروجی

برنامه شما باید دو عدد را در دو خط چاپ کند، در خط اول مدت زمان کل سفر (T) و در خط بعد بیشترین زمان توقف (MaxWait).

مثال

ورودی نمونه ۱

```
3 10
2 5 5 0
6 3 7 1
7 4 6 0
```


خروجی نمونه ۱

17

3

۱. تعداد چراغ $N = 3$ ، طول مسیر $L = 10$ کیلومتر.

۲. چراغ ۱: فاصله از شروع مسیر باقر ۲ کیلومتر، قرمز ۵ دقیقه، سبز ۵ دقیقه، شروع قرمز ($s=0$)

۳. چراغ ۲: فاصله از شروع مسیر باقر ۶ کیلومتر، قرمز ۳ دقیقه، سبز ۷ دقیقه، شروع سبز ($s=1$)

۴. چراغ ۳: فاصله از شروع مسیر باقر ۷ کیلومتر، قرمز ۴ دقیقه، سبز ۶ دقیقه، شروع قرمز ($s=0$)

ورودی نمونه ۲

3 20

5 3 7 1

15 4 6 0

10 2 8 1

خروجی نمونه ۲

20

0

پیام مخفی

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: متوسط
- طراح: آرتین عضدی‌فر

یک گروه مخفی که اعضای آن در سراسر جهان پخش شده‌اند برای اینکه بتوانند پیام‌های خود را به یکدیگر بدون لو رفتن محتوای آن به همدیگر برسانند تصمیم گرفتند آن‌ها را رمزگذاری کنند. اما چون هیچکدام از اعضا حرفه‌ای نبود فقط توانستند یک روش ابتدایی برای رمزگذاری پیام‌هایی که تنها از حروف $A-Z$, $a-z$ تشکیل شده است طراحی کنند، همچنین در کنار پیام باید دستورالعمل رمزگشایی آن نیز فرستاده شود. قرار است به این تیم کمک کنید و برنامه‌ای بسازید که با دریافت پیام و دستور رمزگشایی، محتوای پیام را به دست بیاورد.

ساختار دستور رمزگشایی خطی شامل حروف R, L, N, P, M و تعدادی عدد ۱ رقمی است. هر حرف به معنای یک دستور خاص است که در ادامه معنای همه آن‌ها توضیح داده شده است. دستورات به ترتیب از چپ به راست روی رشته ورودی اعمال می‌شود تا در نهایت به پیام اصلی برسیم. ممکن است بعد از یک حرف یک عدد یک رقمی بیاید که به این معناست که این دستور باید به آن تعداد تکرار شود. مثلاً $N3RP$ به این معناست که ۳ بار دستور N ، یک بار دستور R و در نهایت یک بار دستور P اجرا شود.

توضیح دستورات:

۱. R : آخرین حرف رشته به ابتدای آن منتقل شود (شیفت به راست)
۲. L : اولین حرف رشته به انتهای آن منتقل شود (شیفت به چپ)
۳. N : آخرین کاراکتر رشته را انتخاب کن و با حرف بعدی آن در حروف الفبای انگلیسی جابجا کن. اگر z بود، با a جایگزین شود.
۴. P : آخرین کاراکتر رشته را انتخاب کن و با حرف قبلی آن در حروف الفبای انگلیسی جابجا کن. اگر a بود با z جایگزین شود.

۵. M : رشته را به دو نیمه مساوی تقسیم کن، ترتیب حروف هر نیمه را برعکس کن و دوباره دو قسمت را به هم بچسبان. اگر طول رشته فرد بود، کاراکتر وسط جا بجا نمی‌شود.

در صورتی که حروفی غیر از حروف ذکر شده بیاید دستورالعمل ایراد دارد و پیام زیر چاپ شود:

The operation is Invalid!

و اجرای برنامه متوقف شود.

تضمین می‌شود پیام ورودی فقط شامل حروف الفبای انگلیسی است.

توجه: به حروف بزرگ و کوچک توجه کنید.

برای درک بهتر توضیح مثال را بخوانید.

ورودی

ورودی شامل دو خط است که خط اول آن پیام مخفی و خط دوم آن دستور رمزگشایی آن است.

خروجی

خروجی تنها شامل یک خط است که محتوای پیام است.

مثال

ورودی نمونه

```
vaLJevnia
R2NML3
```

خروجی نمونه

```
iLoveJava
```

▼ توضیح

ابتدا طبق دستور R2 دو حرف آخر به ابتدای رشته منتقل میشوند و `iavaLJevn` به دست می‌آید. سپس حرف آخر از `n` به حرف بعدی آن در حروف الفبا یعنی `o` تبدیل میشود و `iavaLJevo` به دست می‌آید.

در مرحله بعدی دستور `M` اجرا می‌شود، از آنجایی که طول رشته فرد (۹) است، دو زیر رشته `iava` و `Jevo` هرکدام برعکس می‌شوند و به `aveJ` , `avai` تبدیل میشوند و حرف وسط، `L` نیز دست نخورده باقی می‌ماند و رشته به دست آمده می‌شود `aveJLovei` . در مرحله آخر ۳ شیفت به چپ داریم و رشته نهایی یعنی `iLoveJava` حاصل می‌شود.

تحول زمین

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: سخت
- طراح: مهدی افشاری

به شما یک آرایه یک‌بعدی از اعداد صحیح به نام `terrain` داده شده است که نشان‌دهنده ارتفاع قطعات زمین مجاور هم است، به همراه یک رشته به نام `events` که نشان‌دهنده دنباله‌ای زمانی از رویدادهای هواشناسی و زمین‌شناسی است. این رشته فقط از کاراکترهای `'R'` (باران)، `'E'` (زلزله) و `'M'` (شهاب‌سنگ) تشکیل شده است.

شما باید این رویدادها را به ترتیب شبیه‌سازی کرده و وضعیت نهایی آرایه `terrain` را برگردانید:

۱. `'R'` (باران): یک واحد آب روی هر اندیس می‌بارد. آب فوراً به پایین‌ترین همسایه مجاور که ارتفاع آن اکیداً کمتر است، جریان می‌یابد. اگر آب به یک محلی برسد که ارتفاع هر دو همسایه‌اش بیشتر یا مساوی آن باشد، در آنجا جمع شده و ارتفاع آن قطعه را به‌طور دائم ۱ واحد افزایش می‌دهد. اگر آب از لبه‌های چپ یا راست آرایه به بیرون سرازیر شود، ناپدید می‌گردد.

- آپدیت ارتفاع گودال‌ها باید در انتها و همزمان انجام شود
- قطره آب فقط یک قدم به همسایه مجاور نمی‌رود؛ بلکه آنقدر روی شیب به سمت پایین سرازیر می‌شود تا به یک گودالی برسد یا به دیواره آرایه رسیده و از آن سرازیر بشود.

۲. `'E'` (زلزله): همه "قله‌ها" در زمین به‌طور همزمان فرو می‌ریزند. قله به هر قطعه‌ای گفته می‌شود که اکیداً بلندتر از هر دو همسایه چپ و راست خود باشد (لبه‌ها نمی‌توانند قله باشند). یک قله در حال ریزش، ۱ واحد از ارتفاع خود را از دست می‌دهد و هر دو همسایه مجاور آن ۱ واحد ارتفاع به دست می‌آورند.

- افزایش ارتفاع قطعه‌ها در اثر ریزش قله مجاور باید در انتها و همزمان انجام شود.

۳. `'M'` (شهاب‌سنگ): یک شهاب‌سنگ به مرتفع‌ترین قطعه در کل زمین برخورد می‌کند (در صورت برابری، به چپ‌ترین آن‌ها برخورد می‌کند). ارتفاع قطعه مورد اصابت فوراً نصف می‌شود (با استفاده از تقسیم

صحیح). این برخورد یک موج انفجار ایجاد می‌کند!

موج انفجار: شما باید ترتیب عناصر را در یک زیرآرایه که دقیقاً k اندیس به سمت چپ و k اندیس به سمت راست محل برخورد امتداد دارد، معکوس (reverse) کنید؛ که در آن k دقیقاً برابر با مقدار ارتفاع از دست‌رفته قطعه مورد اصابت است. اگر شعاع معکوس‌سازی از مرزهای آرایه فراتر رفت، به سادگی عناصر را تا همان مرز معکوس کنید. (مثال رو ببینید تا بهتر متوجه بشید):

فرمت ورودی

در اولین خط، تعداد عناصر آرایه terrain داده می‌شود. در خط بعد عناصر آرایه terrain با یک فاصله از هم داده می‌شود. در نهایت هم در خط بعد رشته events داده می‌شود.

▼ نکته آموزشی

برای دریافت رشته و تبدیل آن به آرایه ای از کاراکتر ها بدین صورت عمل میکنیم:

```
Scanner sc = new Scanner(System.in);
sc.nextLine(); // دریافت عناصر آرایه
String s = sc.nextLine();
char[] events = s.toCharArray();
```

فرمت خروجی

شما باید آرایه نهایی terrain را در یک خط چاپ کنید

مثال

ورودی

```
5
3 1 5 2 4
REM
```

خروجی

4 5 4 2 3

توضیح

مرحله ۱: رویداد 'R' (باران) قانون: روی هر ۵ اندیس یک قطره آب می‌بارد و به سمت پایین‌ترین همسایه سرازیر می‌شود تا در یک گودال (حداقل محلی) جمع شود. هر قطره، ۱ واحد به ارتفاع گودال اضافه می‌کند.

• **قطره روی اندیس ۰ (ارتفاع ۳):** همسایه سمت راستش ۱ است. آب به سمت پایین (اندیس ۱) می‌رود.

• **قطره روی اندیس ۱ (ارتفاع ۱):** هر دو همسایه‌اش (۳ و ۵) بلندتر هستند. این یک گودال است، پس آب همین‌جا می‌ماند.

• **قطره روی اندیس ۲ (ارتفاع ۵):** همسایه‌هایش ۱ و ۲ هستند. پایین‌ترین آن‌ها ۱ است، پس آب به سمت چپ (اندیس ۱) سرازیر می‌شود.

• **قطره روی اندیس ۳ (ارتفاع ۲):** همسایه‌هایش ۵ و ۴ هستند. این هم یک گودال است، آب همین‌جا می‌ماند.

• **قطره روی اندیس ۴ (ارتفاع ۴):** همسایه سمت چپش ۲ است. آب به سمت پایین (اندیس ۳) سرازیر می‌شود.

نتیجه باران: اندیس ۱ در مجموع ۳ قطره آب جمع کرد (ارتفاع جدید: $1 + 3 = 4$). اندیس ۳ در مجموع ۲ قطره آب جمع کرد (ارتفاع جدید: $2 + 2 = 4$).

• **وضعیت زمین پس از باران:** $[3, 4, 5, 4, 4]$

(وضعیت نهایی ارتفاعات در آخر و به صورت یکجا اعمال میشود)

مرحله ۲: رویداد 'E' (زلزله) قانون: قله‌ها (خانه‌هایی که از هر دو همسایه چپ و راست بلندترند) ۱ واحد فرو می‌ریزند و به هر دو همسایه ۱ واحد اضافه می‌کنند.

• روی زمین فعلی $[3, 4, 5, 4, 4]$ ، تنها یک قله وجود دارد: **اندیس ۲ (ارتفاع ۵)** که از ۴ و ۴ بلندتر است.

• اندیس ۲ فرو می‌ریزد (می‌شود ۴). همسایه‌هایش یعنی اندیس ۱ و اندیس ۳ هر کدام ۱ واحد قد می‌کشند (می‌شوند ۵).

• وضعیت زمین پس از زلزله: [3, 5, 4, 5, 4]

مرحله ۳: رویداد 'M' (شهاب‌سنگ) قانون: برخورد به بلندترین نقطه (در صورت برابری، چپ‌ترین نقطه). ارتفاع نصف می‌شود و به اندازه ارتفاع از دست‌رفته، موج انفجار باعث معکوس شدن (Reverse) آرایه در آن شعاع می‌شود.

• روی زمین فعلی [3, 5, 4, 5, 4]، بیشترین ارتفاع ۵ است. چون دو تا ۵ داریم، چپ‌ترین آن‌ها یعنی اندیس ۱ هدف قرار می‌گیرد.

• ارتفاع اندیس ۱ نصف می‌شود (تقسیم صحیح): $5 / 2 = 2$.

• مقدار ارتفاع از دست‌رفته (شعاع موج انفجار): $5 - 2 = 3$ واحد.

• زمین در لحظه برخورد (قبل از موج): [3, 2, 4, 5, 4]

• **موج انفجار:** باید زیرآرایه‌ای که ۳ خانه به چپ و ۳ خانه به راست محل برخورد (اندیس ۱) کشیده شده را معکوس کنیم.

◦ حد چپ: $-2 = 1 - 3$ (چون از آرایه بیرون می‌زند، همان اندیس ۰ را در نظر می‌گیریم).

◦ حد راست: $4 = 1 + 3$ (دقیقاً تا انتهای آرایه).

• پس باید کل آرایه [3, 2, 4, 5, 4] از اندیس ۰ تا ۴ معکوس شود.

• وضعیت نهایی زمین: [4, 5, 4, 2, 3]

آرایه نهایی: [4, 5, 4, 2, 3]

پله میخوره به کجا؟! (امتیازی)

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: امتیازی
- طراح: سعید خزاعی

تابعی پله‌ای بنام $G(x)$ روی بازه $[1, n]$ تعریف شده است که مقادیر آن فقط ۰ یا ۱ است. می‌دانیم این تابع صعودی است (یعنی اگر $G(a)$ برابر ۱ باشد، برای هر $b > a$ نیز $G(b)$ برابر ۱ خواهد بود). شما می‌خواهید کوچکترین x را پیدا کنید که $G(x)$ برابر ۱ باشد. برای محاسبه مقدار تابع در هر نقطه، هزینه‌ای پرداخت می‌کنید. اگر مقدار تابع در نقطه انتخابی ۱ باشد، یک واحد از اعتبار k شما کسر می‌شود و اگر اعتبار به صفر برسد دیگر نمی‌توانید مقداری را بپرسید. حداقل هزینه‌ای که تضمین کند این x پیدا می‌شود چقدر است؟

ورودی

ورودی تنها شامل یک خط است که در آن دو عدد طبیعی k و n با فاصله از هم آمده است.

$$1 \leq n \leq 1\,000\,000\,000$$

$$1 \leq k \leq 100$$

خروجی

حداقل هزینه را با توجه به صورت سوال چاپ کنید.

مثال

ورودی نمونه ۱

خروجی نمونه ۱

10

ورودی نمونه ۲

2 10

خروجی نمونه ۲

4

توضیح نمونه 2:

برای شناسایی x ، اولین عددی که چک می‌کنیم، عدد 4 است.

دو حالت بوجود می‌آید، اگر خروجی تابع 1 نباشد، باید اعداد بالاتر را چک کنیم، اگر 1 باشد، باید اعداد پایین تر را چک کنیم.

در حالتی که 1 باشد، به حالت نمونه 1 خواهیم رسید. (باید تمامی اعداد بازه را از کوچک به بزرگ چک کنیم.)

در حالتی که خروجی 0 باشد، به سراغ 7 می‌رویم، اگر خروجی تابع برای 7، 1 باشد، باید اعداد بین 4 و 7 را تک به تک چک کنیم. اما اگر خروجی 0 باشد، به سراغ 9 می‌رویم و اگر 9 هم خروجی را 0 کند، با بررسی 10، عدد موردنظر را پیدا می‌کنیم، و اگر 9 خروجی را 1 کند، با بررسی شماره 8، عدد مورد نظر را پیدا می‌کنیم. (توجه کنید که می‌خواهیم اولین عددی که تابع را یک می‌کند پیدا کنیم.)

پس حداقل هزینه ای که نیاز داریم تا به خواسته برسیم، 4 است.